SOFA Project Handover Document AppComm CRM



CRM System

Customer Relation Management Application

Semester 7

Software Factory project (2023-2024)

Date: 12-01-2024

Nils Winking, Maxi Zellmer, Moritz Frieling , Anatol Pavlitschenko & Sjong Leow

Abstract

The SOFA Project CRM is a testament to the successful collaboration between our development team and AppComm to streamline customer data management. This comprehensive Customer Relation Management (CRM) system consolidates disparate data streams into one centralized tool, providing a unified repository for essential customer information, communication details, and a comprehensive inventory of products and services.

Throughout the development process, our team employed robust methodologies such as test-driven development and bi-weekly sprint cycles facilitated by Jira. The use of Laravel and Vue.js, along with PostgreSQL for data storage, reflects our commitment to delivering a scalable and efficient solution. The success of the project is underscored by the implementation of core functionality, including CRUD operations, clean and simple forms, searchable dashboards, and a unified data flow.

Looking ahead, integration with AppComm's existing systems represents an exciting avenue for future development. The recommendations outlined aim to strengthen the CRM system's capabilities and ensure its adaptability to evolving business needs. As we hand over SOFA Project CRM to AppComm, we are grateful for the journey together and are confident that this tool will play a key role in improving customer relation management application.

Content

Abstract 1
1. Project Overview: 4
1.1. Overview: 4
2. Objectives and Scope: 5
2.1. Initial Objectives 5
2.2. Achieved Objectives 5
2.3. Scope of the project 5
3. Technical Overview: 6
3.1. Programming Language(s) Used: PHP 6
3.2. Frameworks and Libraries: Laravel, Vue.js 6
3.3. Database Systems PostgreSQL for testing services 7
4. Architecture: 8
4.1 System Architecture 8
4.2 Database Schema 9
4.3 Third-party Services Used 9
5. Development Process: 10
5.1. Methodology: 10
5.2. Project management tool: 10
5.3. Version control: 10
5.4. Coding practices: 10
5.5. Testing Procedures: 10
5.6. Collaboration and communication: 11
5.7. Project milestones: 11
6. UI/UX Design: 13
6.1 Design Principles: 13
6.2 Tools Used: 13
7. Challenges we faced: 15
Overview of Challenges Encountered 15
8. GitHub Repository: 16
8.1. Repository URL 16
8.2. Branching Strategy 16
9. Setup and Deployment: 17
9.1. Dockerized Setup 17
9.2. Localized Setup 17
9.3. Development 19

9.4. Testing 19
9.5. For coverage: 19
9.6. Usage 19
9.7. GitLab projects 20
10. Outstanding Issues: 21
10.1. Any Pending Bug Fixes or Issues 21
10.2. Features Yet to be Implemented 21
11. Recommendations: 22
11.1. Suggestions for Improvement 22
11.2. Areas for Future Development 22
12. Conclusion: 24
13. References: 25

1. Project Overview:

Welcome to the Project Overview of AppComm CRM System. This document serves as a comprehensive guide, providing stakeholders and team members with a detailed understanding of the project's conception, execution, and outcomes.

In the following sections, we will navigate through key aspects of the project, providing insight into its objectives, scope, technical architecture, development process, and more. Throughout its duration, the project team has overcome challenges, implemented strategic solutions, and laid the groundwork for future enhancements.

1.1. Overview:

- ✓ SOFA project title: AppComm CRM System
- ✓ Project duration:
  - Start date: September 2023
  - End date: December 2023
- ✓ Project Team Members:
  - Nils Winking
  - Maxi Zellmer
  - Moritz Frieling
  - Anatol Pavlitschenko
  - Sjong Leow
- ✓ Stakeholders: AppComm
  - Rudolf Rigtering
  - Barham Majid

Whether you are a project team member, a stakeholder, or an interested party, this document is intended to capture the essence of AppComm CRM System and its significance in a broader context.

2. Objectives and Scope:

The SOFA Project CRM system emerges as a transformative solution, offering a streamlined approach to manage and organize diverse customer data. This Customer Relationship Management (CRM) tool acts as a centralized hub, consolidating vital customer information, communication records, and a comprehensive catalog of products and services utilized by our valued clientele.

2.1. Initial Objectives

At the beginning of the project, the primary goal was to design and develop a sophisticated CRM system tailored to AppComm. Focused on improving customer relationship management, the project aimed to deliver a comprehensive solution that would seamlessly integrate with AppComm's unique requirements.

2.2. Achieved Objectives

Through careful planning and execution, the project successfully met its fundamental objectives. A robust CRUD (Create, Read, Update, Delete) functionality was implemented, enabling the system to efficiently manage customer data. This achievement laid the foundation for an intuitive and user-friendly interface that facilitates seamless interaction with customer information. We even managed to go a bit out of scope and added some more functionality like the ticket system and Gitlab projects.

2.3. Scope of the project

The scope of the project extends to include CRUD operations, a fundamental aspect of effectively managing customer information. The system provides a versatile platform that allows users to create, retrieve, update, and delete data, providing a comprehensive suite of customer data management tools.

3. Technical Overview:

In this section we delve into the technical intricacies that power SOFA Project CRM, providing a comprehensive look at the tools, languages and frameworks used. From the core programming languages, PHP, to frameworks like Laravel and Vue.js, each element contributes to the robustness and scalability of the system. Database systems, version control, and third-party services form the backbone of our technical architecture. This overview serves as a guide to the technology that underpins the CRM, ensuring a clear understanding of the project's technical underpinnings.

3.1. Programming Language(s) Used: PHP

At the core of the SOFA Project CRM system is PHP, a robust and versatile programming language. PHP provides the foundation for building dynamic and feature-rich web applications and is the main programming language that AppComm is using thus making it an ideal choice for developing our CRM solution. Even though it is the main programming language that AppComm is using we still did some research and found some pros and cons.

Pros of PHP:

1. Easy to learn and use: PHP is known for its simplicity and ease of learning. It has a low barrier to entry for new developers. So even if you do not have any experience, it should be fairly easy to learn.
2. Large community and support: PHP has a large and active community. This means there are plenty of resources, tutorials, and community-driven support available. Also, AppComm works with this programing language and can probably if you ask help you learn, give tips, and trick or give you materials to learn PHP.
3. Cross-Platform Compatibility: PHP is platform-independent, making it compatible with various operating systems such as Windows, Linux, and MacOS.
4. Scalability: PHP is scalable and can handle large numbers of concurrent users. Many popular websites, including Facebook, are built with PHP.
5. Integration capabilities: PHP easily integrates with various databases such as MySQL, PostgreSQL, and others, enhancing its capabilities for data-driven applications.

Cons of PHP

1. Inconsistencies in function names: PHP has a large number of functions, but there can be inconsistencies in naming conventions, which can cause confusion for developers.
2. Lack of modern features: Compared to newer languages, PHP initially lacked some modern programming features, although recent versions have introduced improvements.
3. Security concerns: PHP has faced security concerns in the past, and poorly written PHP code can be vulnerable to security threats. However, following best practices can mitigate these risks.
4. Performance: While PHP is suitable for many applications, it may not perform as well as some other languages, especially for certain types of complex or CPU-intensive tasks.
5. Not ideal for large-scale enterprise applications: Although PHP is scalable, some developers argue that it may not be as suitable as other languages for very large and complex enterprise-level applications.

(Parker, 2023)

3.2. Frameworks and Libraries: Laravel, Vue.js

Harnessing the power of Laravel and Vue.js, our development team has leveraged two cutting-edge frameworks to shape the SOFA Project CRM system. Laravel, a PHP web application framework, facilitates elegant and efficient code structuring, while Vue.js, a progressive JavaScript framework,

enhances the user interface with dynamic and interactive components. These frameworks and Libraries are also used in AppComm, we still did some research and found some pros and cons.

Pros of Laravel:

1. Elegant syntax: Laravel has an expressive and elegant syntax that makes the code more readable and developer friendly.
2. Modular Packaging System: Laravel has a modular packaging system that allows developers to break their project into small, manageable modules.
3. Artisan Console: The built-in Artisan console provides powerful tools for tasks such as migrations, database seeding and testing.
4. ORM (Eloquent): Laravel comes with Eloquent, an advanced implementation of the Active Record Pattern that makes database interactions clean and straightforward.
5. Blade Templating Engine: Blade, Laravel's templating engine, simplifies the integration of template files and improves the overall front-end development experience.
6. Laravel Mix: Laravel Mix simplifies the compilation and management of assets, making it easy to work with CSS and JavaScript.

Cons of Laravel:

1. Learning curve: For developers new to Laravel, there may be a learning curve due to the framework's extensive features and capabilities.
2. Performance Overhead: While Laravel provides robust features, it may introduce some performance overhead compared to more lightweight frameworks.

(Guru staff, 2021)

Pros of Vue.js:

1. Progressive framework: Vue.js is designed to be incrementally adaptable, allowing developers to adopt its features incrementally, increasing flexibility.
2. Virtual DOM: Vue.js uses a virtual DOM that optimizes rendering by updating only what is needed, resulting in improved performance.
3. Component-based architecture: Vue.js follows a component-based architecture, making it easier to manage and reuse components throughout the application.
4. Easy Integration: Vue.js is easy to integrate with existing projects. It can be as simple as adding a script tag to an HTML file.
5. Reactivity: Vue.js facilitates bi-directional data binding, ensuring automatic updates between the model and the view, providing a reactive user interface.

Cons of Vue.js:

1. Smaller ecosystem: Compared to larger frameworks like React, Vue.js has a smaller ecosystem, which may result in fewer third-party libraries and components.
2. Maturity: While Vue.js is mature, it may not have the same level of corporate backing and community support as frameworks like React.

(Tobiasz, 2022)

3.3. Database Systems PostgreSQL for testing services

During the development phase, the system relies on PostgreSQL as the database system of choice for testing services. PostgreSQL provides a robust and extensible platform for handling data, ensuring a stable environment for testing and validation. Our approach includes future integration with AppComm's existing systems for seamless data storage. Strict decoupling of the data layer from the business logic is essential for this integration.

4. Architecture:

In this chapter, we delve deep into the intricacies of our project's architecture, exploring the system's structural design, database schema, and the invaluable third-party services that contribute to its functionality.

4.1 System Architecture

This section unravels the system architecture of SOFA Project CRM.
Laravel follows the Model-View-Controller (MVC) architectural pattern, which is a system architecture commonly used in web application development. Here's an overview of each component in the Laravel MVC architecture:

Model (M):

- Represents the data and business logic of the application.
- Interacts with the database, retrieves and stores data.
- Laravel Eloquent is the default ORM (Object-Relational Mapping) used for defining and interacting with database models.

View (V):

- Handles the presentation logic and user interface.
- Receives data from the controller and renders it to the user.
- Laravel uses the Blade templating engine for creating views, allowing developers to write templates with PHP code.

Controller (C):

- Manages the flow of data between the model and the view.
- Processes user requests, interacts with the model to retrieve or manipulate data, and then passes that data to the view for rendering.
- Controllers in Laravel are responsible for handling HTTP requests and returning responses.

(Hanif, 2023)

In addition to the MVC pattern, Laravel incorporates other architectural concepts and design patterns to promote modularity, maintainability, and scalability such as:

- **Service Providers:** Laravel uses service providers to bind services into the container, register services, and perform other bootstrapping tasks.
- **Middleware:** Middleware provides a mechanism to filter HTTP requests entering the application. It can be used for tasks such as authentication, logging, or modifying the HTTP request/response.
- **Routing:** Laravel's routing system allows developers to define application endpoints and map them to controllers and actions.
- **Eloquent ORM:** Laravel's Eloquent ORM is based on the Active Record pattern, providing a simple and expressive way to interact with databases.
- **Dependency Injection:** Laravel supports dependency injection throughout the framework, making it easy to inject dependencies into controllers, services, and other classes.

(RIVO AGENCY, 2023)

By embracing these concepts and patterns, Laravel promotes a clean and organized structure for building modern web applications, making it easier for developers to understand, extend, and maintain their codebase.

4.2 Database Schema

A data schema is a blueprint or structural representation of a database that defines the organization, structure, and relationships of the data stored in the database. It outlines the tables, fields, data types, constraints, and relationships between different entities within the database. Schemas provide a standardized way to organize and manage data, ensuring consistency and integrity. (Kopecky, 2023) Laravel provides a powerful ORM and migration system for defining and managing the database schema. PHP, as a server-side scripting language, is used to execute SQL queries and interact with databases. Vue.js, on the other hand, focuses on building dynamic user interfaces and relies on backend technologies like Laravel and PHP for data interactions and database schema management.

4.3 Third-party Services Used

During the development of this project, we incorporated several third-party services to optimize various aspects of our workflow. These services played a critical role in ensuring efficiency, security, and collaboration throughout the development lifecycle.

Database Service: PostgreSQL

PostgreSQL was selected for testing purposes and served as our database service of choice due to its robust and scalable features. This managed database solution provided essential features such as scalability to accommodate growing data, regular backups for data preservation, and robust security measures to protect sensitive information.

Version Control System: GitHub

The choice of GitHub as version control system was based on their reputation as industry-standard platforms for managing source code repositories. These platforms offered an easy-to-use interface, robust collaboration features, and effective version control capabilities. GitHub enabled seamless integration of code changes, efficient collaboration among team members, and systematic versioning and organization of the project's source code.

5. Development Process:

In the dynamic realm of software development, the success of a project hinges on robust methodologies, collaborative tools, and meticulous procedures.

5.1. Methodology:

We followed the agile principles, with a focus on iterative progress and close collaboration with the customer, AppComm. We used a test-driven approach to ensure the robustness and functionality of the implemented features.

5.2. Project management tool:

Jira serves as the primary project management tool, providing structure, organization, and milestones. Bi-weekly sprints, including planning, reviews, and retrospectives with AppComm, ensure efficient progress, monitored by a sprint burndown chart.

5.3. Version control:

GitHub, served as the version control system. This facilitated seamless collaboration among developers, ensuring version history tracking, code review processes, and overall codebase integrity.

5.4. Coding practices:

Adherence to coding standards and practices was maintained throughout the project. Regular code reviews were conducted to promote knowledge sharing and maintain a high standard of code quality. The code practices we used were:

1. Descriptive Naming: use meaningful and descriptive names for variables, functions, and classes to enhance code readability.
2. Comments and Documentation: Include concise and informative comments to explain complex sections of code. Generate comprehensive documentation for functions and APIs.
3. Code Reviews: Conduct regular code reviews within the development team to catch issues, share knowledge, and maintain code quality.
4. Testing: Adhere to test-driven development (TDD) principles by writing unit tests before implementing functionality. Regularly run automated tests to catch regressions.
5. Code Consistency: Enforce a consistent coding style across the project. This includes conventions for naming, spacing, and formatting.
6. Code Reusability: Design code with reusability in mind to minimize redundancy and make components adaptable for different parts of the application.

(Bose, 2023)

5.5. Testing Procedures:

For the SOFA Project CRM system, we used local feature testing and adopted a test-driven approach. This involved creating tests for specific features before implementing the corresponding functionality, ensuring early detection of issues and alignment with pre-defined requirements.

To streamline our development workflow and maintain code quality, we leveraged the capabilities of GitHub Actions, a robust continuous integration (CI) service that is seamlessly embedded into our GitHub repository. GitHub Actions allowed us to articulate workflows as code, providing an organized and automated framework for various development tasks such as building, testing, and deploying our codebase. Our configured workflows were designed to trigger in response to key events in our development cycle, including code pushes, pull requests, and branch merges. This careful setup not only automated critical tasks, but also contributed to a standardized and reliable development pipeline. In addition, our commitment to code quality included implementing a review process that

required changes to be thoroughly reviewed and approved before being merged into the main or development branches. This additional layer of review ensured that our code base adhered to established standards and remained cohesive throughout the development lifecycle.
As a result, the integration of GitHub Actions into our workflow, combined with a rigorous code review process, played a critical role in improving the overall quality, reliability, and collaborative nature of SOFA Project's CRM system development.

5.6. Collaboration and communication:

In the realm of successful development processes, fostering effective communication and collaboration stands out as a fundamental key. The project team recognized the critical role of effective communication channels throughout the development lifecycle.
To ensure a good and immediate communication, the team used various channels, such as WhatsApp, to ensure quick and efficient exchanges. In addition, regular online meetings via Teams, this provided a platform for virtual face-to-face interactions, fostering a sense of connection despite not being there in person. In the early stages of the project, the team also went for a face-to-face meeting, which laid a solid foundation for collaboration. To emphasize the structured nature of the development cycle, the project team adhered to a biweekly schedule of sprint planning and review meetings. These recurring meetings served as critical touchpoints, allowing the team to collectively address challenges, review progress, and reflect on the overall development process. The commitment to regular, well-organized meetings ensured that issues were identified and resolved in a timely manner, contributing to the overall success of the project.

5.7. Project milestones:

Milestones were consistently met, with successful releases marking the completion of major features. The iterative development process ensured that feedback from each release was incorporated into subsequent iterations. Only one time a project milestone was not met, and we extended the split with one week to adjust and expand on the project. To be able to be so successful you need to be able to do the following things:

- Define clear milestones: Clearly define specific, measurable, and achievable milestones for your project. Each milestone should represent a significant accomplishment or completion of a project phase.
- Create a Detailed Project Plan: Develop a comprehensive project plan that outlines tasks, dependencies, resources, and schedules. Break the project into smaller, more manageable tasks to facilitate tracking.
- Set Realistic Timelines: Set realistic timelines for each milestone. Consider factors such as task complexity, resource availability, and potential risks. Ensure that deadlines are challenging yet achievable.
- Allocate Resources Effectively: Assign resources, including team members and equipment, according to the requirements of each milestone. Ensure that team members have the skills and support they need to fulfill their responsibilities.
- Establish communication channels: Promote clear communication channels within the team. Regularly update team members on project progress, challenges, and changes in plans. Open communication encourages collaboration and problem solving.
- Monitor and Track Progress: Implement a robust tracking system to monitor progress against established milestones. Use project management tools, dashboards, or reports to visualize progress and identify potential delays.
- Identify and address risks: Proactively identify risks that could impact milestone achievement. Develop mitigation strategies and contingency plans. Promptly address risks to prevent delays.

- Celebrate Achievements: acknowledge and celebrate the successful completion of milestones. Recognition boosts morale and motivates the team to maintain momentum toward subsequent project goals.
- Review and Adjust Regularly: Conduct regular reviews of the project plan and milestones. Assess whether adjustments are needed based on changes in scope, resources, or external factors. Be flexible and adapt to changing project dynamics.

(Thai, 2022)

With these steps we successful released and completed major features. The iterative development process ensured that feedback from each release was incorporated into subsequent iterations.

6. UI/UX Design:

User Interface (UI) and User Experience (UX) design are integral aspects of any software development project, playing a pivotal role in shaping how users interact with and perceive a product. UI/UX design goes beyond creating aesthetically pleasing interfaces; it encompasses the entire user journey, ensuring a seamless, intuitive, and enjoyable experience. (Lamprecht, 2023)

6.1 Design Principles:

The design focused on simplicity, with an intuitive user interface.

Simplicity:
Objective: Prioritize a clean and uncluttered interface.
Implementation: Streamlined navigation, minimalist design elements, and straightforward user interactions.

Consistency:
Objective: Maintain consistency throughout the application.
Implementation: Standardized design patterns, consistent color schemes, and coherent layout structures.

User-centric approach:
Objective: Align design with the needs and expectations of end users.
Implementation: Conducted user research, gathered feedback, and iteratively refined the interface for optimal user satisfaction.

Accessibility:
Objective: Ensure accessibility for users of all abilities.
Implementation: Utilized accessible design practices, provided alternative text for images, and adhered to color contrast guidelines.

Together, these design principles guided the development team in creating a UI/UX that not only met the functional requirements, but also provided a simple yet enjoyable and efficient experience for the end user.

6.2 Tools Used:

ChatGPT
Using ChatGPT as a valuable tool in this endeavor has significantly enriched the content, allowing for more nuanced explanations, detailed insights, and a more streamlined documentation process. Using this advanced language model improves the overall clarity and coherence of the report, code and many more things.

Docker
For our project, we chose Docker as our testing tool because of its unparalleled efficiency in creating consistent and reproducible test environments. Docker's containerization technology allows us to encapsulate applications and their dependencies, ensuring that each test instance is isolated and maintains identical configurations. This choice was motivated by the need for reliable and scalable testing practices that align with continuous integration and delivery (CI/CD) principles. Docker's ability to streamline the testing process, eliminate inconsistencies between development and test environments, and accelerate test cycles significantly influenced our decision. The versatility and ease of container sharing also played a critical role in improving collaboration among team members,

making Docker a strategic choice for maintaining testing consistency and reliability throughout our project.

Figma
Figma is a collaborative interface design tool for creating interactive and visually appealing user interfaces. It allows multiple users to collaborate in real time, making it an excellent choice for team-based design projects. Figma provides a cloud-based platform that enables seamless sharing, commenting, and updating of design files, promoting efficient communication among team members.

We chose Figma as our wireframing tool because of its collaborative nature and ease of use. The real-time collaboration features facilitated effective communication and collaboration among team members, streamlining the wireframing process. The cloud-based platform ensured accessibility and version control, allowing our team to work on wireframes simultaneously, ultimately increasing the efficiency and cohesiveness of our UI/UX design workflow. Wireframes can be found in our GitHub repository.

Jira
Choosing Jira as our primary project management tool was a strategic decision driven by its comprehensive features and easy-to-use interface. Jira excels at facilitating meticulous planning, tracking and management of projects, and offers robust support for various methodologies, including agile practices such as Scrum. The platform's versatility seamlessly aligns with our project needs, providing an integrated solution that enhances collaboration and streamlines workflows. Jira's intuitive nature, coupled with its powerful features, made it the ideal choice to ensure effective project management and successful project outcomes.

PHPStorm (IDE)
PHPStorm is an integrated development environment (IDE) designed specifically for PHP web development. It offers a robust set of features, including code completion, debugging, version control integration, and support for various PHP frameworks. The IDE provides a user-friendly interface and advanced coding support, making it a popular choice for PHP developers.

We chose PHPStorm for its comprehensive PHP development tools and features tailored to increase productivity and code quality. The IDE's intelligent coding assistance, seamless integration with version control systems, and support for various frameworks fit well with the needs of our PHP-based project. PHPStorm's user-friendly interface and extensive customization options contribute to a smooth and efficient development experience, making it the preferred IDE for our team. The only downside is that you will need a license to access PHPStorm, but if you are a student you can request it for free. You can find it here https://www.jetbrains.com/community/education/#students

PlantUML
We made all our diagrams by using PlantUML, which we choose for its ability to streamline the creation of UML diagrams directly from textual descriptions. PlantUML is a versatile tool for creating Unified Modeling Language (UML) diagrams from textual descriptions. It allows developers to express complex system architectures, processes, and designs in a visual and standardized way. PlantUML's strength lies in its simplicity and ease of integration into various documentation processes.

7. Challenges we faced:

In the project we faced some challenges in this chapter we will delve into these challenges and describe what we did to overcome them.

Overview of Challenges Encountered

Learning new Languages and Frameworks

Learning new languages and frameworks like Laravel and Vue.js can be challenging but breaking it down into manageable steps and adopting effective strategies can make the process smoother. Here are some tips that helped us when we started so maybe it will work for you as well:

1) Understand the basics: before diving into frameworks, make sure you have a solid understanding of the underlying languages. For Laravel, understand PHP, and for Vue.js, be comfortable with JavaScript.
2) Official documentation: start with the official documentation. Laravel and Vue.js provide comprehensive and well-structured documentation that serves as an excellent learning resource.
3) Projects and hands-on practice: Apply your knowledge by working on Laravel project. You can generate a standard Laravel project and mess around with it. This hands-on experience is critical for reinforcing concepts and gaining practical skills.
4) Ask for help: take advantage of the wealth of knowledge available to you by contacting the AppComm team. Their extensive experience makes them a valuable resource for overcoming challenges and accelerating your understanding of Laravel and Vue.js. Don't hesitate to seek their expertise when you hit a roadblock in your learning journey.
5) Have patience: pace yourself. Learning a new framework takes time, so be patient and consistent in your efforts and you will be able to learn and eventually master them all.

Communication and agreements

At the earlier stages of our teamwork with AppComm we had some misunderstandings this led to some rework or double work. We have taken several steps to proactively prevent these issues.

1) Clear communication channels: establish transparent communication channels to effectively disseminate information. Such as Teams, Discord, Slack WhatsApp etc.
2) Regular Updates: Schedule regular updates to keep all team members informed of project progress and changes.
3) Detailed Documentation: Maintain comprehensive documentation to serve as a reference and guide for everybody involved. This can be reference and checked to avoid misunderstandings.
4) Feedback Loops: ask for feedback on the documentation, you can even let them sign so that in the future everyone can see what you agreed on and that there are no misunderstandings.

We did not encounter any additional challenges throughout the project. Collaboration within the team and with AppComm was extremely smooth and any issues that arose were dealt with promptly and effectively.

8. GitHub Repository:

8.1. Repository URL

Here is the GitHub repository link: https://github.com/FontysVenlo/sofa-project-chatrobot_appcomm
This is a private repository and you need ask AppComm or one of the lectures at Fontys to give you permission to view this repository.

8.2. Branching Strategy

A branching strategy is a systematic approach to managing branches in a version control system, such as Git in the context of GitHub. It defines rules and guidelines for creating, merging, and organizing branches within a repository. The primary goal is to facilitate collaboration, code stability, and efficient release management.
The branching strategy that we chose is feature branching. Each new feature or bug fix is developed in a separate branch, promoting isolation and parallel development. When the feature is complete, it's merged into the main branch. But before that is possible, we also set that at least one other member of the team needs to review for it before it is merged in to the main branch. This will help you maintain code quality, preventing bugs from sneaking in your code, keep it consistent and overall help the team grow and learn from each other.

9. Setup and Deployment:

What is described here can also be found in the GitHub README file but maybe this version has a bit more explanation and a step by step how to setup, install and deploy. The Sofa Project AppComm CRM system is fully dockerized, you typically won't need to install anything on your local machine. Docker containers encapsulate the application and its dependencies.

9.1. Dockerized Setup

When using Docker, you would typically follow these steps:

1. Clone the Repository:

git clone https://github.com/FontysVenlo/sofa-project-chatrobot_appcomm.git

```bash
git clone https://github.com/FontysVenlo/sofa-project-chatrobot_appcomm.git
```

2. Build and Start Docker Containers:

docker-compose up

```bash
docker-compose up
```

3. Initialize the Database:

docker exec crmapp-crm-1 php artisan migrate
docker exec crmapp-crm-1 php artisan key:generate
docker exec crmapp-crm-1 php artisan db:seed --class=AdminUserSeeder

```bash
docker exec crmapp-crm-1 php artisan migrate
docker exec crmapp-crm-1 php artisan key:generate
docker exec crmapp-crm-1 php artisan db:seed --class=AdminUserSeeder
```

4. Access the Application:

The application should be available at the specified URL, often http://localhost:8000

Always refer to the specific project documentation or README file for accurate and project-specific instructions.

9.2. Localized Setup

If you want to run it locally then you need to install some components after cloning the repository:

Step 1:

composer install
npm install

```bash
composer install
npm install
```

Once the dependencies are installed, copy the .env.example file to .env and fill in the required values, especially the database connection information. Run the following commands to set up the database and generate the application key:

Step 2:
php artisan migrate
php artisan key:generate

```bash
php artisan migrate
php artisan key:generate
```

Refer to the "Default Admin User" section if you want to create an admin user automatically. Afterward, start the development server with:

Step 3:
npm run build
php artisan serve

```bash
npm run build
php artisan serve
```

The application should be accessible at http://localhost:8000.

Default admin user
During installation, an admin user can be automatically generated. If you use the provided .env.example, an admin user is created with the following credentials:

Step 4:
Name: Admin
Email: admin@example.com
Password: password

To create a different admin user after the installation, replace the values in the .env file before running the migration. After changing the values, run the following command:

Step 5:
php artisan db:seed --class=AdminUserSeeder

```bash
php artisan db:seed --class=AdminUserSeeder
```

For sample data:

Step 6:
php artisan db:seed --class=CustomerSeeder
php artisan db:seed --class=SolutionSeeder

```bash
php artisan db:seed --class=CustomerSeeder
php artisan db:seed --class=SolutionSeeder
```

9.3. Development

For development, it's recommended to run the following commands for automatic recompilation:

npm run dev
php artisan serve

```bash
npm run dev
php artisan serve
```

9.4. Testing

Run tests with:
php artisan test

```bash
php artisan test
```

9.5. For coverage:

php artisan test –coverage

```bash
php artisan test --coverage
```

Create a .env.testing file for different environment variables for testing. Afterward, migrate the testing database:
php artisan migrate --env=testing

```bash
php artisan migrate --env=testing
```

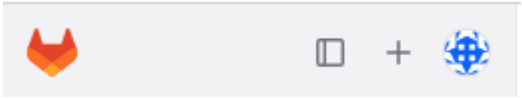
9.6. Usage

After installation, log in with either the admin user you created or the default admin user.

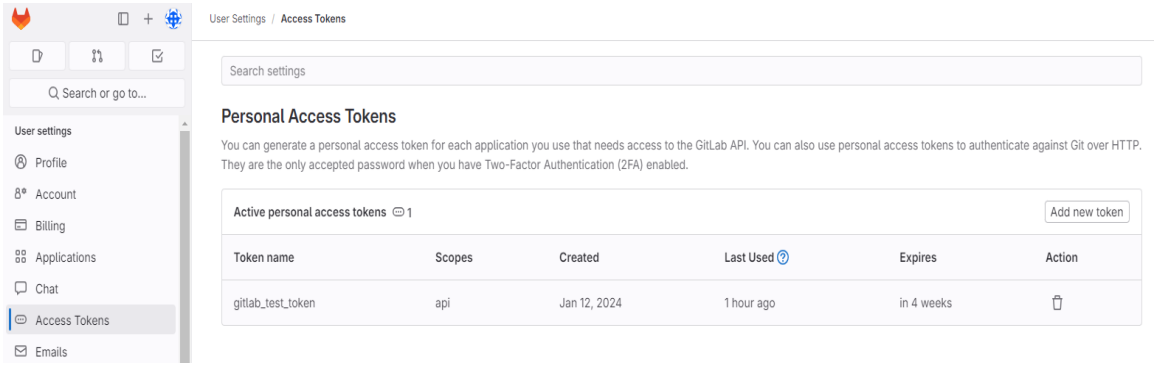
9.7. GitLab projects

To be able to add your GitLab project to a customer you first need to add your Personal Access Token from your GitLab account to the env. file, you can follow the following steps to do that.
Go to your GitLab account, if you don't have one then create one first, then you can create new projects or import them from your GitHub if you want.

To make the token, click on your profile picture, it is located at the top left, the orange one is the GitLab logo (home) and the blue one is your profile (this can be different depending on your account).



click on preferences, go to access tokens, add a new token there, you just have to check the API and then generate the token.



After you have generated the token, you need to add it to the env file. you need to add your token at the end of GITLAB_TOKEN=.
For example:

GITLAB_TOKEN= "your personal access token here”

Once all this is done and added, then you will be able to add GitLab projects to your customers.

10. Outstanding Issues:

In this section, we will address any outstanding issues or unresolved matters that may require attention or further clarification. The goal is to provide transparency regarding any outstanding concerns.

10.1. Any Pending Bug Fixes or Issues

No significant outstanding issues are present at the culmination of the Sofa project. The team successfully implemented the requested functionality, validated CRUD operations, and conducted extensive testing to guarantee seamless functionality.

10.2. Features Yet to be Implemented

The feature that still needs to be implemented is the implementation is the functionality for GitHub Labs project to filter on names. This was not addressed within the project timeline due to time constraints. It's important to note that while the core functionality, specifically CRUD operations, was successfully implemented, additional enhancements such as filtering GitHub Labs project on names were considered additional features. The prioritization of core functionality took precedence, resulting in this feature being deferred to future development phases.

11. Recommendations:

In the recommendations section, we present valuable insights and suggestions for future improvement or optimizations based on our experience throughout the project. These recommendations are intended to guide the ongoing development and improvement of the CRM system, ensuring its continued effectiveness and relevance.

11.1. Suggestions for Improvement

While we have met the core requirements, we recognize certain enhancements that can further increase the effectiveness of the system. One notable aspect is the implementation of sorting and filtering for GitLab project names, a small but valuable addition, due time constraints it was not possible for us to implement this feature. Going forward, the following recommendations aim to strengthen the CRM system.

Update to the contact management system:
Enhance the current contact management system with more sophisticated organization, filtering, and searching capabilities. This ensures a more robust and efficient handling of customer information.

Workflow Automation:
Integrate with AppComm's Workflow Robot to automate business processes and workflows, reducing manual effort and improving overall efficiency.

Security and customer access:
Enabling customers to access the system by implementing advanced security measures. This involves necessary changes to the security system to ensure customer access without compromising data integrity.

11.2. Areas for Future Development

This section outlines specific features and functionality that, while not implemented within the current project schedule, are identified for potential inclusion in future development cycles. These areas represent opportunities for enhancement and expansion beyond the core features already successfully implemented.

Email Integration:
Integrate with email systems to track communications, schedule follow-ups, and maintain a centralized communication history. This capability adds depth to customer interaction management.

Customer Service and Support:
Implement a comprehensive platform for managing customer support tickets, resolving issues, and tracking customer inquiries. Consider integrating a chatbot or robotic system for preliminary issue resolution.

Analytics and Reporting:
Implement analytics and reporting capabilities to gain insights from customer interactions, sales performance, and other relevant metrics. Data-driven decision making becomes more accessible with these tools.

Mobile accessibility:
Prioritizing accessibility by introducing a mobile platform that grants on the go access to customer information, updates and more. This enhances the overall user experience, making essential data readily available whenever and wherever needed.

Ask the organization/client: AppComm

A well-implemented CRM system can enhance customer satisfaction, improve sales efficiency, and contribute to overall business success by fostering better relationships with customers. The specific features and functionalities needed may vary depending on the organization's size, industry, and business goals so the last area for future development that I can mention is to ask what AppComm's further needs and wishes are to better enhance the CRM.

12. Conclusion:

In conclusion, the successful completion of the SOFA project CRM marks a significant milestone in streamlining customer data management for AppComm. The collaborative efforts of the project team, coupled with effective communication and strategic decision making, resulted in a robust CRM system. The implementation of core functionality, including CRUD operations, clean and simple forms, searchable dashboards, and a unified data flow, ensures an efficient and user-friendly experience.

While the project met its primary objectives, it also identified areas for future development, particularly in integrating with AppComm's existing systems. The use of technologies such as Laravel, Vue.js and PostgreSQL, along with the adoption of Jira for project management and GitHub Actions for continuous integration, reflects a commitment to industry best practices.

As we turn the project over to AppComm, we recognize the ongoing journey of continuous improvement. The recommendations outlined and areas identified for future development provide a roadmap for enhancing the functionality of the CRM system to ensure it remains adaptable to evolving business needs.

We would like to thank the entire project team and AppComm for their collaboration and support throughout the development process. We hope that the SOFA Project CRM will make a significant contribution to AppComm's customer relationship management, and we look forward to its continued success in the future.

13. References:

Bose, S. (2023, April 20). *BrowserStack*. Retrieved from Coding Standards and Best Practices to Follow: https://www.browserstack.com/guide/coding-standards-best-practices

Guru staff. (2021, December 1). *Guru Insight*. Retrieved from The Pros and Cons of Laravel: https://www.guru.com/blog/the-pros-and-cons-of-laravel/

Hanif, F. (2023, June 26). *Medium*. Retrieved from Understanding the MVC Architecture in Laravel: A Comprehensive Guide: https://fkrihnif.medium.com/understanding-the-mvc-architecture-in-laravel-a-comprehensive-guide-8f620cc139b6#:~:text=concept%20in%20Laravel.-,MVC%20Concept%20in%20Laravel,scalable%20and%20maintainable%20web%20applications.

Kopecky, C. (2023, November 20). *Educative*. Retrieved from What are database schemas?: https://www.educative.io/blog/what-are-database-schemas-examples

Lamprecht, E. (2023, June 2). *CareerFoundry*. Retrieved from The Difference Between UX and UI Design: A Beginner's Guide: https://careerfoundry.com/en/blog/ux-design/the-difference-between-ux-and-ui-design-a-laymans-guide/

Parker, A. (2023, August 31). *Ellow*. Retrieved from Advantages and Disadvantages of PHP: https://ellow.io/advantages-and-disadvantages-of-php/

RIVO AGENCY. (2023, July 15). *Medium*. Retrieved from Best practices for developing scalable and maintainable Laravel applications: https://medium.com/@rivo.agency2010/best-practices-for-developing-scalable-and-maintainable-laravel-applications-4d91f684492a

Thai, J. (2022, November 9). *Asana*. Retrieved from How to set and achieve project milestones in 2022: https://asana.com/resources/project-milestones

Tobiasz, F. (2022, April 28). *The Codest*. Retrieved from Using Vue: Pros and Cons: https://thecodest.co/blog/pros-and-cons-of-vue/